🎯

DBMS Unit - 3 (Textbook)

📘 4.1 SQL Data Definition & Data Types

Where databases learn their shape, structure, and soul.

🧱 4.1.1 Schema & Catalog Concepts in SQL

> Imagine SQL as a city. Schemas are neighborhoods, catalogs are entire districts, and INFORMATION_SCHEMA is the all-seeing municipal records office.

🗂️ Schema — the Architectural Blueprint

A **schema** groups together all objects belonging to one application:

- **Tables**
- **Domains**
- **Views**
- **Constraints**
- **Authorization details**

🔑 **Ownership:**

Each schema has an **authorization identifier** — the owner of the neighborhood.

🛠️ **Creation (the spell that creates worlds):**

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

⚠️ **Access Control:**

Not everyone gets to play god. Only users with specific privileges (granted by the DBA) can create schemas.

🏛️ Catalog — the Grand Library of Schemas

A **catalog** is a **collection of schemas** under one SQL environment (i.e., one RDBMS installation).

Every catalog includes:

- **INFORMATION_SCHEMA** → metadata oracle that stores details about *all* schemas & their elements.

🔗 **Important rule:**

- Integrity constraints (like foreign keys) can only link tables **within the same catalog**.
- Schemas in the same catalog can share **domains**.

🏗️ 4.1.2 The CREATE TABLE Command

> The moment where data stops being abstract and becomes real enough to store.

✨ Purpose

`CREATE TABLE` defines:

- Table name
- Attributes
- Data types
- Constraints (NULL/NOT NULL, keys, referential integrity)

🧩 Syntax Vibes

Attributes are declared first — each with:

- **name**
- **data type**

- **constraints**

Example style:

```
CREATE TABLE COMPANY.EMPLOYEE (...);
```

or simply

```
CREATE TABLE EMPLOYEE (...);
```

(if the schema is implied by the environment)

🧱 Base Tables vs Virtual Tables

Concept	Meaning
Base Table	Physically stored in DB; created via `CREATE TABLE`
Virtual Table (View)	Logical table; defined by `CREATE VIEW`; may or may not be stored physically

Note:

- In base tables, attributes are considered **ordered**.
- Rows are **not** considered ordered.

  (A gentle reminder that SQL believes in equality... until ORDER BY enters the room.)

⚠️ Subtle Constraints Chaos

Some `CREATE TABLE` statements can blow up silently if:

- A **foreign key points to a table that hasn't been created yet**
- A **table references itself** (circular reference)

Example:

`Super_ssn` in EMPLOYEE referencing EMPLOYEE → circular.

`Dno` in EMPLOYEE referencing DEPARTMENT → DEPARTMENT must exist first.

Moral?

SQL is like a meticulous librarian — it hates references to books that aren't printed yet.

🧬 4.1.3 Attribute Data Types & Domains in SQL

Where data takes on shape, size, flavour — and a few surprising quirks.

🧊 Overview

SQL offers a variety of data types so each attribute can hold values that match its role.

The major categories are:

- **Numeric**
- **Character Strings**
- **Bit Strings**
- **Boolean**
- **Date & Time**
- **Timestamp & Interval**
- **Large Objects (CLOB, BLOB)**
- **Domains (custom data types)**

Let's unpack them with Notion-clean precision.

🔢 Numeric Data Types

For values that count, measure, and occasionally overflow.

🧱 Integer Types

- **INTEGER / INT**

- **SMALLINT**

🌊 Floating-Point Types

- **FLOAT, REAL, DOUBLE PRECISION**
  - Store approximate numeric values.

📐 Formatted / Fixed Precision Numbers

- **DECIMAL(i, j)** or **NUMERIC(i, j)**
  - **i:** total digits (precision)
  - **j:** digits after decimal (scale)
  - Default scale = 0 (integer behavior)

Example: `DECIMAL(5,2)` → max value `999.99`

Perfect for salaries, prices, and anything requiring financial honesty.

🔤 Character String Types

Words, names, descriptions — packaged neatly.

📌 Fixed-Length

- **CHAR(n)** or **CHARACTER(n)**
  - If shorter, SQL pads with blanks on the right.
  - `'Smith'` in `CHAR(10)` becomes `'Smith '`.

🪢 Variable-Length

- **VARCHAR(n)** or **CHARACTER VARYING(n)**
  - Stores only the actual length.
  - Max length = `n`.

✏️ String Literals

- Always in single quotes `'like this'`

- Case sensitive for data, *not* for SQL keywords.

🔠 Ordering & Comparison

- Alphabetic (lexicographic) order
- Padded blanks ignored in comparisons
- Concatenation operator: `||`
  - `'abc' || 'XYZ'` → `'abcXYZ'`

📚 Large Text

- **CLOB(n)** — Character Large Object
  - Store documents, long texts
  - Size can be in K, M, or G → example: `CLOB(20M)`

💡 Bit String Types

For binary-level detail: flags, masks, encoded data.

⚙️ Fixed-Length

- **BIT(n)**

🔧 Variable-Length

- **BIT VARYING(n)**

🧪 Bit Literals

- Written as: **B'10101'**
- Not to be confused with character strings `'10101'`.

📦 Binary Large Object

- **BLOB(n)** — for large binary data (images, media)
  - Example: `BLOB(30G)`

- Size units: K, M, G (but in *bits*, not bytes)

🧙 Hexadecimal shortcut

- Length multiple of 4 → can use: **X'FA3B'**

🟩 Boolean Data Type

SQL's version comes with an existential crisis:

- **TRUE**
- **FALSE**
- **UNKNOWN** (thanks to NULL values)

This leads to **three-valued logic**, explored deeper in Chapter 5.

🗓️ Date & Time Data Types

SQL's calendar can be strict, but fair.

📅 DATE

Format: **YYYY-MM-DD**

Example: `DATE '2008-09-27'`

⏰ TIME

Format: **HH:MM:SS**

Example: `TIME '09:12:47'`

🕑 Time Precision

- **TIME(i)** → adds fractional seconds
  - Example: `TIME(3)` → `12:30:59.123`

🌍 Time Zones

TIME WITH TIME ZONE

- Adds ±HH:MM offset
- Range: +13:00 to –12:59

🔎 Validity Rules

- Month: 1–12
- Days: 1–31 (only if valid for that month)
- Comparisons: `<`, `>`, etc. work chronologically.

🕰️ Timestamp & Interval

📌 TIMESTAMP

Includes:

- DATE
- TIME
- Fractional seconds
- Optional time zone

Example:

`TIMESTAMP '2008-09-27 09:12:47.648302'`

⏳ INTERVAL

Represents a *relative* time span for adding or subtracting:

- **YEAR / MONTH intervals**
- **DAY / TIME intervals**

Perfect for "deliver this in 3 days" logic.

🎨 Casting & Comparisons

SQL treats DATE, TIME, TIMESTAMP like specialized strings, so they can be cast for comparisons or formatting.

🧱 Domains — Custom Data Types

Your chance to define your own mini universe.

Domains give:

- Reusability
- Uniform constraints
- Cleaner schemas

✨ Example

Creating a Social Security Number type:

```
CREATE DOMAIN SSN_TYPE AS CHAR(9);
```

Now replace all CHAR(9) occurrences with **SSN_TYPE** — beautiful consistency.

Domains can include:

- Data type
- **DEFAULT** value
- **CHECK** constraints (covered later)

```
CREATE TABLE EMPLOYEE
        ( Fname          VARCHAR(15)      NOT NULL,
          Minit          CHAR,
          Lname          VARCHAR(15)      NOT NULL,
          Ssn            CHAR(9)          NOT NULL,
          Bdate          DATE,
          Address        VARCHAR(30),
          Sex            CHAR,
          Salary         DECIMAL(10,2),
          Super_ssn      CHAR(9),
          Dno            INT              NOT NULL,
        PRIMARY KEY (Ssn),
        FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn),
        FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE DEPARTMENT
        ( Dname          VARCHAR(15)      NOT NULL,
          Dnumber        INT              NOT NULL,
          Mgr_ssn        CHAR(9)          NOT NULL,
          Mgr_start_date DATE,
        PRIMARY KEY (Dnumber),
        UNIQUE (Dname),
        FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) );
CREATE TABLE DEPT_LOCATIONS
        ( Dnumber        INT              NOT NULL,
          Dlocation      VARCHAR(15)      NOT NULL,
        PRIMARY KEY (Dnumber, Dlocation),
        FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE PROJECT
        ( Pname          VARCHAR(15)      NOT NULL,
          Pnumber        INT              NOT NULL,
          Plocation      VARCHAR(15),
          Dnum           INT              NOT NULL,
        PRIMARY KEY (Pnumber),
        UNIQUE (Pname),
        FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) );
CREATE TABLE WORKS_ON
        ( Essn           CHAR(9)          NOT NULL,
          Pno            INT              NOT NULL,
          Hours          DECIMAL(3,1)     NOT NULL,
        PRIMARY KEY (Essn, Pno),
        FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),
        FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) );
CREATE TABLE DEPENDENT
        ( Essn           CHAR(9)          NOT NULL,
          Dependent_name VARCHAR(15)      NOT NULL,
          Sex            CHAR,
          Bdate          DATE,
          Relationship   VARCHAR(8),
        PRIMARY KEY (Essn, Dependent_name),
        FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) );
```

Figure 4.1
SQL CREATE TABLE data definition statements for defining the COMPANY schema from Figure 3.7.

🛡️ 4.2 Specifying Constraints in SQL

Where tables learn discipline, values learn boundaries, and chaos is politely escorted out.

SQL constraints ensure data remains **valid**, **consistent**, and **meaningful**.

This section covers the foundational constraints used during table creation.

🎯 4.2.1 Attribute Constraints & Attribute Defaults

Teaching each column how to behave.

🚫 NOT NULL

- Prevents an attribute from holding NULL.
- Automatically applied to **all attributes of the primary key**.
- Can be added to any other attribute where a value is always required.

📦 DEFAULT <value>

- Assigns a **default value** when no explicit value is provided during insertion.
- If no default is declared → default default = **NULL** (unless NOT NULL applies).
- Example (from textbook contexts):
  - Default manager for a new department
  - Default department for a new employee

🔍 CHECK Constraints

Mini rules attached to attributes or domains.

✔️ Attribute-level CHECK

```
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);
```

✔️ Domain-level CHECK

Define once, reuse everywhere:

```
CREATE DOMAIN D_NUM AS INTEGER
CHECK (D_NUM > 0 AND D_NUM < 21);
```

Use **D_NUM** for all department-number attributes across multiple tables — clean, consistent, elegant.

🔑 4.2.2 Key & Referential Integrity Constraints

Ensuring every row has an identity and every reference has a home.

🌟 PRIMARY KEY

- Specifies the attribute(s) forming the primary key.
- Auto-applies NOT NULL.
- Can be placed directly on an attribute:

```
Dnumber INT PRIMARY KEY;
```

🌟 UNIQUE

- Specifies alternate keys (candidate keys).
- Ensures all values in that attribute (or attribute set) are distinct.

```
Dname VARCHAR(15) UNIQUE;
```

🔗 FOREIGN KEY

The relational glue connecting tables.

- Declared using `FOREIGN KEY (...) REFERENCES ...`.
- Violations typically trigger **RESTRICT** (rejecting the offending operation).
- Designers may specify custom actions using clauses:

⭐ Referential Triggered Actions

Action	ON DELETE	ON UPDATE
SET NULL	Set referencing FK to NULL	Set referencing FK to NULL
SET DEFAULT	Set referencing FK to default	Set referencing FK to default
CASCADE	Delete referencing tuples	Propagate new primary key

Example explained:

For foreign key `Super_ssn` in EMPLOYEE:

- **ON DELETE SET NULL** → If supervisor is deleted, employees lose (only) their supervisor's SSN.
- **ON UPDATE CASCADE** → If supervisor's SSN changes, employees' `Super_ssn` updates automatically.

💡 When to use CASCADE

Best suited for:

- Relationship tables (e.g., **WORKS_ON**)
- Multivalued-attribute tables (e.g., **DEPT_LOCATIONS**)
- Weak entities (e.g., **DEPENDENT**)

(And yes, circular FKs like EMPLOYEE.Super_ssn usually need to be added later via ALTER TABLE.)

🏷️ 4.2.3 Naming Constraints

Give your rules names so you can find them later — a librarian's dream.

- Syntax: `CONSTRAINT <constraint_name> ...`
- Names must be **unique within a schema**.
- Useful when altering or dropping constraints later.
- Naming is optional but highly recommended for maintainability.

📐 4.2.4 Tuple-Based Constraints with CHECK

Higher-level checks applied per row.

These appear at the **end of a CREATE TABLE** statement and apply to each tuple.

Example:

If DEPARTMENT includes a creation date, we may ensure the manager starts **after or on** that date:

```
CHECK (Dept_create_date <= Mgr_start_date)
```

CHECK for general constraints

- More powerful, global constraints use:
  `CREATE ASSERTION`
- Covered in Chapter 5 (because they require full query power).

```
CREATE TABLE EMPLOYEE
    (  . . . ,
       Dno        INT            NOT NULL        DEFAULT 1,
    CONSTRAINT EMPPK
       PRIMARY KEY (Ssn),
    CONSTRAINT EMPSUPERFK
       FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
                    ON DELETE SET NULL          ON UPDATE CASCADE,
    CONSTRAINT EMPDEPTFK
       FOREIGN KEY(Dno) REFERENCES DEPARTMENT(Dnumber)
                    ON DELETE SET DEFAULT       ON UPDATE CASCADE);
CREATE TABLE DEPARTMENT
    (  . . . ,
       Mgr_ssn    CHAR(9)        NOT NULL        DEFAULT '888665555',
       . . . ,
    CONSTRAINT DEPTPK
       PRIMARY KEY(Dnumber),
    CONSTRAINT DEPTSK
       UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
       FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
                    ON DELETE SET DEFAULT   ON UPDATE CASCADE);
CREATE TABLE DEPT_LOCATIONS
    (  . . . ,
    PRIMARY KEY (Dnumber, Dlocation),
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)
                    ON DELETE CASCADE           ON UPDATE CASCADE);
```

Figure 4.2
Example illustrating how default attribute values and referential integrity triggered actions are specified in SQL.

📡 4.3 Basic Retrieval Queries in SQL

SELECT: the single, surprisingly deep sentence that asks a database to tell a story.

🔎 Overview — what this section teaches

- **SELECT** is SQL's main retrieval statement (not the same as relational-algebra σ).
- SQL tables are **multisets (bags)** by default — duplicates allowed — unless constrained by keys or `DISTINCT`.
- We'll build from the simple **SELECT–FROM–WHERE** trio up to ordering, pattern matching, and set ops.

🧩 The SELECT–FROM–WHERE structure (the classic mapping block)

Template

```
SELECT <attribute list>
FROM   <table list>
WHERE  <condition>;
```

- **SELECT** → projection attributes (what to show).
- **FROM** → the relations involved.
- **WHERE** → selection & join conditions (who qualifies).

Comparison ops: `=, <, <=, >, >=, <>` (SQL's `<>` = not equal).

🧪 Example queries (short & sweet)

Q0 — *Find John B. Smith's birthdate and address*

```
SELECT Bdate, Address
FROM   EMPLOYEE
WHERE  Fname='John' AND Minit='B' AND Lname='Smith';
```

Q1 — *Employees who work for 'Research' (join DEPARTMENT & EMPLOYEE)*

```
SELECT E.Fname, E.Lname, E.Address
FROM   EMPLOYEE E, DEPARTMENT D
WHERE  D.Dname='Research' AND D.Dnumber = E.Dno;
```

Q2 — *Projects in 'Stafford' with manager info*

```
SELECT P.Pnumber, P.Dnum, M.Lname, M.Address, M.Bdate
FROM   PROJECT P, DEPARTMENT D, EMPLOYEE M
WHERE  P.Dnum = D.Dnumber AND D.Mgr_ssn = M.Ssn AND P.Plocati
on = 'Stafford';
```

Q8 — *Each employee and their immediate supervisor (self-join + aliases)*

```
SELECT E.Fname, E.Lname, S.Fname AS SupFname, S.Lname AS SupL
name
FROM   EMPLOYEE AS E, EMPLOYEE AS S
WHERE  E.Super_ssn = S.Ssn;
```

🏷️ Ambiguous names, aliases & tuple variables

- If attributes repeat across tables, **qualify**: `Table.Attr`.
- Use **aliases** to shorten and clarify: `EMPLOYEE E` or `EMPLOYEE AS E`.
- Aliases make **self-joins** readable (E = employee, S = supervisor).

✳️ No WHERE clause & the asterisk

- **No WHERE** → select **all tuples** from the FROM list.
- Multiple tables with no WHERE → **CROSS PRODUCT** (cartesian explosion).
- selects **all attributes** of the tables listed.

Examples:

```
SELECT Ssn FROM EMPLOYEE;           -- all Ssns
SELECT Ssn, Dname FROM EMPLOYEE, DEPARTMENT;  -- all combinat
ions
SELECT * FROM EMPLOYEE WHERE Dno = 5;         -- all attribut
es for Dno=5
```

⚖️ Tables as multisets vs sets — DISTINCT and ALL

- SQL results are **multisets** unless you use `DISTINCT` (removes duplicates).
- `SELECT ALL` (or omitting ALL/DISTINCT) ⇒ keep duplicates (default).
- Aggregate functions often want duplicates preserved.

Example:

```
SELECT ALL   Salary FROM EMPLOYEE;     -- salaries, duplicate
s shown
SELECT DISTINCT Salary FROM EMPLOYEE;  -- unique salary value
s only
```

🔀 Set operations (work like set math — but watch duplicates)

Operation	Removes duplicates?	Multiset variant
`UNION`	Yes	`UNION ALL` keeps all duplicates
`EXCEPT` / `MINUS`	Yes	`EXCEPT ALL` keeps duplicates
`INTERSECT`	Yes	`INTERSECT ALL` keeps duplicates

Note: Relations must be *union-compatible* (same # and order of attrs).

Example (projects involving any 'Smith' as manager or worker):

```
( SELECT DISTINCT Pnumber
  FROM PROJECT, DEPARTMENT, EMPLOYEE
  WHERE Dnum = Dnumber AND Mgr_ssn = Ssn AND Lname = 'Smith'
)
UNION
( SELECT DISTINCT Pnumber
  FROM PROJECT, WORKS_ON, EMPLOYEE
  WHERE Pnumber = Pno AND Essn = Ssn AND Lname = 'Smith' );
```

🔤 Substring pattern matching — `LIKE`, `%`, `_`

- `%` = any sequence of 0+ characters
- `_` = exactly one character
- Escape special chars with `ESCAPE` and single quotes inside strings as `''`.

Examples:

```
WHERE Address LIKE '%Houston,TX%'     -- contains Houston,TX
WHERE Bdate LIKE '__5_______'       -- born in 195x (example
format)
WHERE Name LIKE 'AB\_CD\%EF' ESCAPE '\'
```

➕ Arithmetic, string concat, BETWEEN, dates & intervals

- Arithmetic: `+ - * /` usable in SELECT/WHERE.
- Concatenate strings with `||`.
- Date/time + interval arithmetic allowed.
- `BETWEEN a AND b` ⇔ `>= a AND <= b`.

Example — 10% raise for ProductX workers:

```
SELECT E.Fname, E.Lname, 1.1 * E.Salary AS Increased_sal
FROM   EMPLOYEE E, WORKS_ON W, PROJECT P
WHERE  E.Ssn = W.Essn AND W.Pno = P.Pnumber AND P.Pname = 'Pr
oductX';
```

Example — salary range:

```
WHERE Salary BETWEEN 30000 AND 40000 AND Dno = 5;
```

↕ ORDER BY — sorting results

- Use `ORDER BY <attr list>` to control display order.
- Default: **ascending (ASC)**. Use `DESC` for descending.

Example:

```
ORDER BY D.Dname, E.Lname, E.Fname;
-- or mix directions:
ORDER BY D.Dname DESC, E.Lname ASC, E.Fname ASC;
```

🧾 Quick cheat-sheet: clauses & order

```
SELECT <attribute list>
FROM   <table list>
[ WHERE <condition> ]
[ ORDER BY <attribute list> ];
```

- Mandatory: `SELECT`, `FROM`.
- Optional: `WHERE`, `ORDER BY`.
- Coming later (Chapter 5): `GROUP BY` and `HAVING`, aggregates, nested queries, advanced joins, recursion.

🛠️ 4.4 INSERT, DELETE & UPDATE — Mutating the Database

When data takes action — rows arrive, vanish, or change their minds.

🪄 4.4.1 INSERT — adding tuples

Purpose: add new tuples (rows) to a table.

✨ Forms

1. **Positional INSERT (values in table order)**

```
INSERT INTO EMPLOYEE
VALUES ('Richard','K','Marini','653298653','1962-12-30',
        '98 Oak Forest, Katy, TX','M',37000,'653298653',4);
```

- Values must be in the **same order** as attributes in the `CREATE TABLE`.
- DBMS enforces constraints (keys, NOT NULL, FK) — invalid insert → **rejected**.

1. **Column-list INSERT (sparse or partial inserts)**

```
INSERT INTO EMPLOYEE (Fname, Lname, Dno, Ssn)
VALUES ('Richard','Marini',4,'653298653');
```

- Attributes **not listed** get **DEFAULT** or **NULL** (unless NOT NULL → error).
- Order of values follows the **INSERT column list**, not table order.

1. **Multi-row INSERT**

```
INSERT INTO T (a,b) VALUES (1,2), (3,4), (5,6);
```

1. **INSERT ... SELECT** — create/populate from query result

```
CREATE TABLE WORKS_ON_INFO (Emp_name VARCHAR(15), Proj_name V
ARCHAR(15), Hours_per_week DECIMAL(3,1));
INSERT INTO WORKS_ON_INFO (Emp_name, Proj_name, Hours_per_wee
k)
SELECT E.Lname, P.Pname, W.Hours
FROM PROJECT P, WORKS_ON W, EMPLOYEE E
WHERE P.Pnumber = W.Pno AND W.Essn = E.Ssn;
```

- Handy for snapshots; **not** automatically kept up-to-date (use a **VIEW** for live join results).

⚠️ Practical tips

- If FK or NOT NULL constraints are violated, the DBMS **rejects** the insert.
- Use column-list INSERT to avoid errors when table schema changes or has many columns.

🗑️ 4.4.2 DELETE — removing tuples

Purpose: remove tuples from a single relation.

🔎 Syntax & semantics

```
DELETE FROM EMPLOYEE WHERE Lname = 'Brown';   -- delete selec
ted rows
DELETE FROM EMPLOYEE;                          -- delete all
rows, table remains
```

- **WHERE** filters which tuples to delete. No WHERE → all tuples removed.
- Only deletes from the **named table**. It does **not** drop table definition (use `DROP TABLE` for that).

🧩 Referential behavior

- If **referential actions** (ON DELETE CASCADE, SET NULL, etc.) are defined on foreign keys, deletions can **propagate** automatically to referencing tables.
- If action is RESTRICT (default), delete is **rejected** when referential integrity would break.

Examples from text (effects)

- `DELETE` may delete zero, one, many, or all tuples depending on WHERE.

🔁 4.4.3 UPDATE — changing tuples

Purpose: modify attribute values of selected tuples in one relation.

🧾 Basic form

```
UPDATE PROJECT
SET Plocation = 'Bellaire', Dnum = 5
WHERE Pnumber = 10;
```

- `WHERE` chooses which rows to update.
- `SET` lists attributes to be changed and their new expressions.

🔁 Use old values to compute new ones

```
UPDATE EMPLOYEE
SET Salary = Salary * 1.1
WHERE Dno = 5;
```

- Right-hand `Salary` is old value; left-hand is new — in-place arithmetic update.

⚠️ Referential & constraint notes

- Updating a **primary key** may cascade to referencing foreign keys **if** `ON UPDATE CASCADE` is defined.
- Updates that violate **NOT NULL**, **CHECK**, **UNIQUE**, or **FK** constraints are **rejected**.
- Each `UPDATE` affects **one table** at a time; update multiple tables with multiple statements.

🧭 Quick-career DBA tips (practical, skeptical, and kind)

- Always prefer **column-list INSERT** in production to avoid silent bugs when schema changes.
- Be explicit with transaction boundaries (`BEGIN` / `COMMIT`) when doing multi-step data mutations — snapshots can bite.
- Beware of accidental full-table updates/deletes — try `SELECT` with the same WHERE before mutating.
- Use **referential actions** mindfully:
  - `CASCADE` is wonderful for relationship tables, dangerous for parent tables with many dependents.
  - `SET NULL` preserves dependents but loses the link — useful for soft unlinking.
- When creating derived snapshot tables (INSERT ... SELECT), document that they are **static** unless refreshed or replaced by views.

📚 5.1 More Complex SQL Retrieval Queries

When SELECT learns recursion, existential questions, and the strange logic of NULL.

🧩 5.1.1 Comparisons Involving NULL & Three-Valued Logic

SQL treats **NULL** as a mysterious wildcard — not "empty," not "zero," but *unknown / unavailable / not applicable*.

Since NULL refuses to commit to TRUE or FALSE, SQL behaves wisely and uses a **three-valued logic**:

- **TRUE**
- **FALSE**
- **UNKNOWN** ← the troublemaker

🌀 What NULL Can Mean

Meaning	Example
Unknown	DOB not known
Withheld	Has phone, but doesn't want to share
Not applicable	LastCollegeDegree for someone with no degree

SQL **does not distinguish** between these meanings — NULL is NULL.

🎛️ Three-Valued Logic Truth Tables

(a)

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

(b)

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

(c)

NOT	
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

🔍 Checking for NULL

SQL **never** uses `= NULL` or `<> NULL`.

Instead use:

```
IS NULL
IS NOT NULL
```

Example — employees without supervisors:

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Super_ssn IS NULL;
```

🧱 5.1.2 Nested Queries, Tuples & Set/Multiset Comparisons

Sometimes you need a query *inside* another query — a database Inception moment.

A **nested (sub)query** lives inside:

- WHERE
- FROM
- SELECT (rare)

⭐ IN Operator

Evaluates TRUE if a value is found in a **set/multiset**.

Example (from reformulated Q4A):

```
SELECT DISTINCT Pnumber
FROM PROJECT
WHERE Pnumber IN (SELECT Pnumber ... )
   OR Pnumber IN (SELECT Pno ... );
```

🧊 Scalar Subqueries

If the inner query returns **one value**, you can use:

```
=, <, >, <=, >=
```

instead of IN.

🧩 Tuple Comparisons

SQL allows comparing multi-attribute tuples:

```
WHERE (Pno, Hours) IN (
    SELECT Pno, Hours
    FROM WORKS_ON
```

```
    WHERE Essn='123456789'
);
```

This finds employees who work the **same (project, hours)** combination as employee 123456789.

🎯 ANY / SOME / ALL Operators

Operator	Meaning
`= ANY` (or `= SOME`)	Equivalent to IN
`> ANY`	Greater than *at least one* value
`> ALL`	Greater than *every* value

Example — employees whose salary exceeds all salaries in department 5:

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE Salary > ALL (
      SELECT Salary FROM EMPLOYEE WHERE Dno=5
);
```

Equivalent to comparing with `MAX(Salary)`.

⚠️ Scope & Ambiguity in Nested Queries

Unqualified attributes refer to the **innermost query's FROM clause**.

Example Q16 — tricky because both EMPLOYEE and DEPENDENT have `Sex`:

Query 16. Retrieve the name of each employee who has a dependent with the same first name and is the same sex as the employee.

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE E.Ssn IN (
```

```
    SELECT D.Essn
    FROM DEPENDENT AS D
    WHERE E.Fname = D.Dependent_name
      AND E.Sex = D.Sex      -- must qualify E.Sex
);
```

Safe practice:

Always give aliases to tables in complex queries.

🔁 5.1.3 Correlated Nested Queries

A nested query becomes *correlated* when it depends on values from the **outer** query.

Think of it as a loop:

> For each tuple in the outer query, evaluate the inner query.

Example — Q16 revisited:

Outer tuple: one employee

Inner query: fetch dependents with same name & sex

If match exists → select employee

Equivalent non-nested formulation (Q16A):

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.Ssn = D.Essn
  AND E.Sex = D.Sex
  AND E.Fname = D.Dependent_name;
```

Correlated queries can often be rewritten as joins, but the nested form expresses the logic more intuitively.

🧭 5.1.4 EXISTS & UNIQUE — existential checks and uniqueness tests

When SQL asks "is there at least one?" or "is this result unique?"

🔎 EXISTS(Q)

- **Returns TRUE** if the nested query `Q` produces **at least one tuple**.
- **Typically correlated**: `Q` refers to attributes from the outer query.
- Use `NOT EXISTS(Q)` to test emptiness.

Why use EXISTS?

- Efficient for "does a related row exist?" checks — the DB can stop after finding one match.

Example — correlated EXISTS (Q16B)

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE AS E
WHERE EXISTS (
  SELECT * FROM DEPENDENT AS D
  WHERE E.Ssn = D.Essn
    AND E.Sex = D.Sex
    AND E.Fname = D.Dependent_name
);
```

> For each employee E: does at least one dependent D match? If yes → include E.

Query 6. Retrieve the names of employees who have no dependents.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS (
  SELECT * FROM DEPENDENT
```

```
  WHERE Ssn = Essn
);
```

Query 7. List the names of managers who have at least one dependent.

```
Q7:   SELECT   Fname, Lname
      FROM     EMPLOYEE
      WHERE    EXISTS ( SELECT   *
                        FROM     DEPENDENT
                        WHERE    Ssn=Essn )
               AND
               EXISTS ( SELECT   *
                        FROM     DEPARTMENT
                        WHERE    Ssn=Mgr_ssn );
```

🧾 UNIQUE(Q)

- **TRUE** if `Q` returns **no duplicate tuples** (i.e., result is a set).
- Useful to test whether a nested query returns unique values.

🔁 5.1.4 — Universal quantification via NOT EXISTS + EXCEPT

"Works on all projects of dept 5" — expresses 'for all' using set-difference emptiness.

Approach 1 — EXCEPT + NOT EXISTS (Q3A)

The query Q3: *Retrieve the name of each employee who works on* all *the projects controlled by department number 5* can be written using EXISTS and NOT EXISTS in SQL

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE NOT EXISTS (
```

```
  (SELECT Pnumber FROM PROJECT WHERE Dnum = 5)
  EXCEPT
  (SELECT Pno FROM WORKS_ON WHERE Ssn = Essn)
);
```

- If `(projects_of_dept5) - (projects_employee_works_on)` is empty → employee works on all.

Approach 2 — nested NOT EXISTS logic (Q3B)

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE NOT EXISTS (
  SELECT * FROM WORKS_ON B
  WHERE B.Pno IN (SELECT Pnumber FROM PROJECT WHERE Dnum = 5)
    AND NOT EXISTS (
      SELECT * FROM WORKS_ON C
      WHERE C.Essn = Ssn AND C.Pno = B.Pno
    )
);
```

- More procedural: "there does not exist a project p of dept 5 such that employee does NOT work on p."

🧾 5.1.5 Explicit sets & renaming (aliasing)

🟢 Explicit set example (Q17)

Query 17. Retrieve the Social Security numbers of all employees who work on project numbers 1, 2, or 3.

```
SELECT DISTINCT Essn
FROM WORKS_ON
```

```
WHERE Pno IN (1, 2, 3);
```

🏷️ Alias / AS for readability (Q8A)

Query 8. For each employee, retrieve the employee's first and last name and the first and last name of his or her immediate supervisor.

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

- Use `AS` to rename **attributes** or **relations** in results — great for column headers and clarity.

🔗 5.1.6 Joined Tables & Outer Joins — joining with intention

JOIN syntax lets you declare how tables should meet — elegantly, or awkwardly.

🧩 Joined table basic (explicit JOIN)

```
SELECT Fname, Lname, Address
FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)
WHERE Dname = 'Research';
```

- `JOIN` (or `INNER JOIN`) → include only matching pairs (inner join).

🌿 NATURAL JOIN

- Implicit equijoin on attributes with **same names**; duplicates collapsed (matching attrs appear once).
- Can rename attrs to make NATURAL JOIN work if names differ.

Example using renaming (Q1B):

```
FROM (EMPLOYEE NATURAL JOIN
      (DEPARTMENT AS DEPT (Dname, Dno, Mssn, Msdate)))
```

🟦 OUTER JOIN variants

- **LEFT OUTER JOIN** (or `LEFT JOIN`): include *all* left-table rows; nonmatches padded with NULLs for right attrs.
- **RIGHT OUTER JOIN**: symmetric for the right table.
- **FULL OUTER JOIN**: include all rows from both sides; nonmatches padded with NULLs.
- `OUTER` keyword is optional in many dialects.

Example — include employees even if they have no supervisor (Q8B):

```
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM (EMPLOYEE AS E LEFT OUTER JOIN EMPLOYEE AS S
      ON E.Super_ssn = S.Ssn);
```

CROSS JOIN / Cartesian product

- `CROSS JOIN` = all combinations.
- Danger: huge results if you forget join conditions. Use only when you mean it.

🧱 Nested / multiway joins

You can nest joins to build multi-table joined tables:

```
FROM ((PROJECT JOIN DEPARTMENT ON Dnum = Dnumber)
      JOIN EMPLOYEE ON Mgr_ssn = Ssn)
```

(Example: Q2A)

🛠️ Dialect footnote & legacy syntaxes

- Some systems (older/Oracle-style) use operators like `+=`, `=+` to express outer joins (e.g., `E.Super_ssn += S.Ssn`).
- Prefer explicit `LEFT/RIGHT/FULL OUTER JOIN` for portability and clarity.

🧮 5.1.7–5.1.8 — Aggregate Functions, GROUP BY & HAVING

Summaries, groups, and the rules that let SQL sing choruses instead of reciting every solo.

✨ What aggregates do

Aggregate functions **compress many rows into a single summary value** for a group (or for the whole result):

- `COUNT(...)` — how many (rows or non-NULL values)
- `SUM(...)` — sum of numeric values
- `MAX(...)` — largest (works for any totally ordered domain)
- `MIN(...)` — smallest
- `AVG(...)` — arithmetic mean

Important behaviors

- Aggregate functions **ignore NULLs** (except `COUNT(*)`, which counts rows regardless of NULLs).
- Use `DISTINCT` inside some aggregates to remove duplicates (e.g., `COUNT(DISTINCT Salary)`).
- `MAX` / `MIN` work on strings/dates if those domains are totally ordered.

🔢 Simple aggregation examples

Query 19. Find the sum of the salaries of all employees, the maximum salary, the minimum salary, and the average salary.

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)
FROM EMPLOYEE;
```

Query 20. Find the sum of the salaries of all employees of the 'Research' department, as well as the maximum salary, the minimum salary, and the average salary in this department.

```
SELECT SUM(Salary), MAX(Salary), MIN(Salary), AVG(Salary)
FROM EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber
WHERE Dname = 'Research';
```

Queries 21 and 22. Retrieve the total number of employees in the company (Q21) and the number of employees in the 'Research' department (Q22).

```
SELECT COUNT(*) FROM EMPLOYEE;
```

Q22 — count in Research:

```
SELECT COUNT(*)
FROM EMPLOYEE, DEPARTMENT
WHERE Dno = Dnumber AND Dname = 'Research';
```

Query 23. Count the number of distinct salary values in the database.

```
SELECT COUNT(DISTINCT Salary) FROM EMPLOYEE;
```

function, and then use the nested query in the WHERE clause of an outer query. For example, to retrieve the names of all employees who have two or more dependents (Query 5), we can write the following:

```
SELECT Lname, Fname
FROM EMPLOYEE
WHERE (SELECT COUNT(*) FROM DEPENDENT WHERE Ssn = Essn) >= 2;
```

🧩 GROUP BY — partition, then summarize

`GROUP BY` partitions result rows into groups that share the same value(s) of the grouping attribute(s). Aggregates are then computed per group.

Query 24. For each department, retrieve the department number, the number of employees in the department, and their average salary.

```
SELECT Dno, COUNT(*), AVG(Salary)
FROM EMPLOYEE
GROUP BY Dno;
```

- Each output row = one department group.
- If grouping attribute value is `NULL`, those rows form their own group.

Query 25. For each project, retrieve the project number, the project name, and the number of employees who work on that project.

```
SELECT Pnumber, Pname, COUNT(*)
FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno
GROUP BY Pnumber, Pname;
```

Rule: Every non-aggregate expression in `SELECT` must appear in `GROUP BY` (or be functionally dependent on it).

🧾 HAVING — filter whole groups

`WHERE` filters **rows before** grouping. `HAVING` filters **groups after** aggregation.

Query 26. For each project *on which more than two employees work*, retrieve the project number, the project name, and the number of employees who work on the project.

```
SELECT Pnumber, Pname, COUNT(*)
FROM PROJECT, WORKS_ON
WHERE Pnumber = Pno
GROUP BY Pnumber, Pname
HAVING COUNT(*) > 2;
```

Query 27. For each project, retrieve the project number, the project name, and the number of employees from department 5 who work on the project.

```
SELECT Pnumber, Pname, COUNT(*)
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE Pnumber = Pno AND Ssn = Essn AND Dno = 5
GROUP BY Pnumber, Pname;
```

⚠️ Common pitfall (WHERE vs HAVING)

You want: **For departments that have >5 employees, count how many make >40k**.

Wrong (filters rows first then groups):

```
-- WRONG: this restricts tuples to Salary>40000 before groupi
ng
SELECT Dname, COUNT(*)
FROM DEPARTMENT, EMPLOYEE
WHERE Dnumber = Dno AND Salary > 40000
```

```
GROUP BY Dname
HAVING COUNT(*) > 5;
```

This returns departments that have **>5 employees earning >40k**, not "departments with >5 employees, and (for those) count how many earn >40k."

Correct approach — use a nested query (or compute groups twice). Example using `IN` with grouped subquery:

```
SELECT Dnumber, COUNT(*)
FROM DEPARTMENT, EMPLOYEE
WHERE Dnumber = Dno
  AND Salary > 40000
  AND Dnumber IN (
    SELECT Dno
    FROM EMPLOYEE
    GROUP BY Dno
    HAVING COUNT(*) > 5
  )
GROUP BY Dnumber;
```

Explanation: inner subquery finds departments with >5 employees (grouping on all employees). Outer query counts only >40k earners in those departments.

🧭 Tips, rules & mini-checklist

- `COUNT(*)` counts rows; `COUNT(attr)` counts non-NULL values of `attr`.
- Aggregates ignore `NULL` values (so watch `AVG`, `SUM`).
- `SELECT` may only contain:
  - grouping attributes, and/or
  - aggregate expressions.
- `HAVING` can use aggregates; `WHERE` cannot. If you want to filter groups by aggregate, use `HAVING`.

- Use `DISTINCT` inside aggregates when you need to ignore duplicates (e.g., `COUNT(DISTINCT Essn)`).
- When in doubt, break into nested queries — correctness beats cleverness.

🎯 5.1.9 — Discussion & Summary of SQL Queries

Where SQL's many moving parts finally assemble into a clean, humming machine.

🧩 The Six Possible Clauses of an SQL Query

Only the first two are mandatory; the rest join the party if needed.

```
SELECT <attribute / function list>
FROM <table list>
[ WHERE <condition> ]
[ GROUP BY <grouping attributes> ]
[ HAVING <group conditions> ]
[ ORDER BY <attribute list> ];
```

✨ What each clause actually does:

Clause	Purpose
SELECT	Pick the columns or computed values to display.
FROM	Declare all tables (or joins) the query touches.
WHERE	Filter & join rows *before* grouping.
GROUP BY	Partition rows into groups.
HAVING	Filter entire groups based on aggregate conditions.
ORDER BY	Sort the final output.

🧠 Conceptual Evaluation Order

(Not how DBMS actually runs it, but how SQL thinks about it.)

1️⃣ **FROM**

- Materialize tables
- Apply join logic

2️⃣ **WHERE**

- Row-level filtering
- Only tuples evaluating to TRUE survive

3️⃣ **GROUP BY**

- Split surviving rows into groups

4️⃣ **HAVING**

- Filter groups based on aggregate conditions

5️⃣ **SELECT**

- Output required attributes and aggregate results

6️⃣ **ORDER BY**

- Sort the final answer ✨

> Think of SQL like a chef:
>
> **FROM** gathers ingredients →
>
> **WHERE** discards the bad ones →
>
> **GROUP BY** plates them nicely →
>
> **HAVING** rejects plates that don't meet the standards →
>
> **ORDER BY** arranges the final presentation.

🔁 Many ways to write the *same* query

SQL is flexible — perhaps *too* flexible.

You can express the same logic using:

- joins in WHERE

- JOIN syntax in FROM
- nested queries
- EXISTS, IN, ANY/ALL
- correlated subqueries

Pros: freedom, expressiveness, comfort.

Cons: confusion, inconsistent performance across DBMSs.

> Your query is a poem; the DBMS is the critic.
>
> Some structures read better — and some run faster.

🧱 5.4 — Schema Change Statements in SQL

Because databases, like cities, evolve. New roads appear; old buildings fall.

🔥 5.4.1 The DROP Command

Used to delete schema elements entirely: tables, domains, constraints, or whole schemas.

🧨 DROP SCHEMA

```
DROP SCHEMA COMPANY CASCADE;
```

Drop behaviors:

Option	Meaning
CASCADE	Remove the schema *and everything inside it*: tables, constraints, views.
RESTRICT	Only drop if nothing references it. DBMS will refuse otherwise.

🗑️ DROP TABLE

```
DROP TABLE DEPENDENT CASCADE;
```

- Removes **table + definition**
- BREAKS all referencing constraints unless CASCADE is used
- More destructive than DELETE (which removes rows, not structure)

When to use what:

Goal	Command
Delete rows, keep table	`DELETE`
Delete table + definition	`DROP TABLE`

🔧 5.4.2 The ALTER Command

The surgeon's toolkit for modifying a living schema.

You can:

- Add a column
- Drop a column
- Change column definition
- Add/remove constraints
- Change defaults

🏗️ Add a new column:

```
ALTER TABLE COMPANY.EMPLOYEE
ADD COLUMN Job VARCHAR(12);
```

- Existing rows get **NULL** unless DEFAULT is specified
- Therefore NOT NULL is not allowed during creation unless default exists

---

🧹 Drop a column:

```
ALTER TABLE COMPANY.EMPLOYEE
DROP COLUMN Address CASCADE;
```

- **CASCADE** removes dependent constraints & views
- **RESTRICT** blocks if anything depends on that column

📝 Modify defaults:

```
ALTER TABLE COMPANY.DEPARTMENT
ALTER COLUMN Mgr_ssn DROP DEFAULT;

ALTER TABLE COMPANY.DEPARTMENT
ALTER COLUMN Mgr_ssn SET DEFAULT '333445555';
```

🔒 Drop or Add Constraints

You can only drop constraints *if they have names*.

```
ALTER TABLE COMPANY.EMPLOYEE
DROP CONSTRAINT EMPSUPERFK CASCADE;
```

Then add a new constraint:

```
ALTER TABLE COMPANY.EMPLOYEE
ADD CONSTRAINT ...
```

Table 5.2 Summary of SQL Syntax

CREATE TABLE <table name> (<column name> <column type> [<attribute constraint>] { , <column name> <column type> [<attribute constraint>] } [<table constraint> { , <table constraint> }])
DROP TABLE <table name> ALTER TABLE <table name> ADD <column name> <column type>
SELECT [DISTINCT] <attribute list> FROM (<table name> { <alias> } | <joined table>) { , (<table name> { <alias> } | <joined table>) } [WHERE <condition>] [GROUP BY <grouping attributes> [HAVING <group selection condition>]] [ORDER BY <column name> [<order>] { , <column name> [<order>] }]
<attribute list> ::= (* | (<column name> | <function> (([DISTINCT] <column name> | *))) { , (<column name> | <function> (([DISTINCT] <column name> | *)) }))
<grouping attributes> ::= <column name> { , <column name> }
<order> ::= (ASC | DESC)
INSERT INTO <table name> [(<column name> { , <column name> })] (VALUES (<constant value> , { <constant value> }) { , (<constant value> { , <constant value> }) } | <select statement>)
DELETE FROM <table name> [WHERE <selection condition>]
UPDATE <table name> SET <column name> = <value expression> { , <column name> = <value expression> } [WHERE <selection condition>]
CREATE [UNIQUE] INDEX <index name> ON <table name> (<column name> [<order>] { , <column name> [<order>] }) [CLUSTER]
DROP INDEX <index name>
CREATE VIEW <view name> [(<column name> { , <column name> })] AS <select statement>
DROP VIEW <view name>

NOTE: The commands for creating and dropping indexes are not part of standard SQL.

🔗 15.2 Functional Dependencies

The mathematical compass behind database normalization.

📘 15.2.1 What is a Functional Dependency (FD)?

Imagine your entire database as one huge “universal” relation:

```
R = {A1, A2, A3, …, An}
```

Even though we *never* store data this way, this universal perspective helps define theoretical constraints.

🎯 Definition (Core Idea)

A **functional dependency** between two attribute sets **X → Y** means:

> In every legal instance of R,
>
> if two tuples agree on **X**,
>
> they *must* also agree on **Y**.

You can think of it as:

- **X determines Y**
- **Y depends on X**
- Knowing X → we know Y uniquely

🧠 Semantics Over Syntax

FDs are not discovered by looking at a table *once*.

They come from the **meaning** of attributes.

> FDs describe truths about the real-world relationships the database models — not accidents of one sample dataset.

For example:

- `{State, Driver_license_number} → Ssn`

  always true for U.S. adults.

- `Zip_code → Area_code`

  used to be true decades ago but no longer holds.

FDs reflect *rules of reality*, not sample data patterns.

🔑 Keys Are Special FDs

If **X** is a **candidate key**, then:

```
X → R          (X determines all attributes)
```

A key guarantees no two tuples share the same X value, so X determines everything.

🧭 FD Direction Matters

FDs are **not symmetric**.

If:

```
X → Y
```

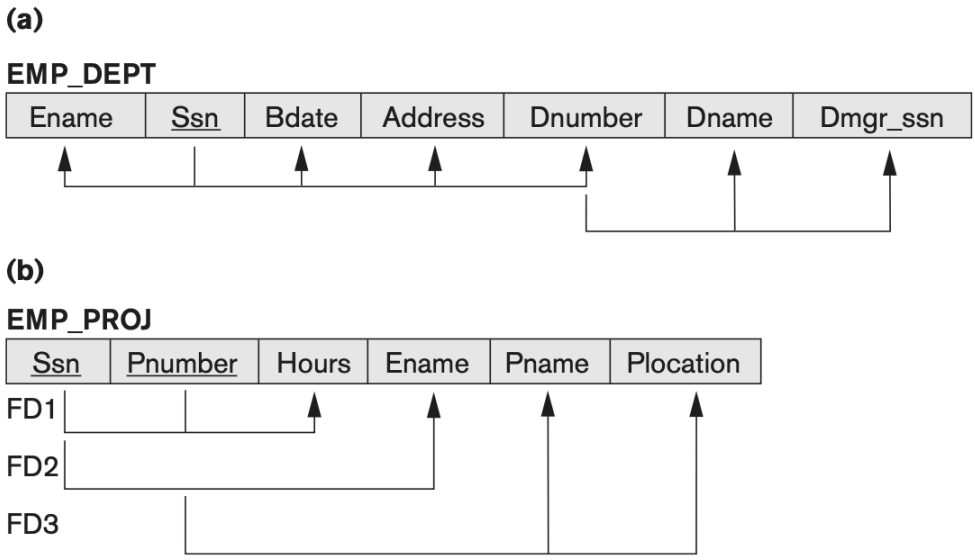
This **does not imply**:

```
Y → X
```

Unless the meaning of attributes truly enforces it.

🧱 Example: EMP_PROJ Schema

From the semantics of an employee–project relationship:



✔️ These FDs must hold:

1. **Ssn → Ename**
2. **Pnumber → {Pname, Plocation}**
3. **{Ssn, Pnumber} → Hours**

Meaning:

- A Social Security number uniquely determines a person's name.
- A Project number uniquely determines its identity.
- A specific employee working on a specific project determines the weekly hours.

🚫 FD Truth Cannot Be Proven From One Table Alone

You **cannot infer** an FD from a single table unless *domain knowledge* confirms it.

Example:

In TEACH relation:

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

We see Smith teaches two different subjects → **Teacher → Course does NOT hold**.

But we can't confirm whether **Text → Course** holds unless we know if that must be true for *all* possible legal states.

🧪 How to Disprove an FD

One counterexample is enough.

If *any two tuples* violate:

```
t1[X] = t2[X]
but t1[Y] ≠ t2[Y]
```

Then **X → Y** is *false*.

Example in Figure 15.8:

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

FD fails:

- **A → B** (tuples 1 and 2 break it)
- **B → A** (tuples 2 and 3 break it)
- **D → C** (tuples 3 and 4 break it)

FD might hold (no violations):

- **B → C**
- **C → B**
- **{A, B} → C**
- **{A, B} → D**
- **{C, D} → B**

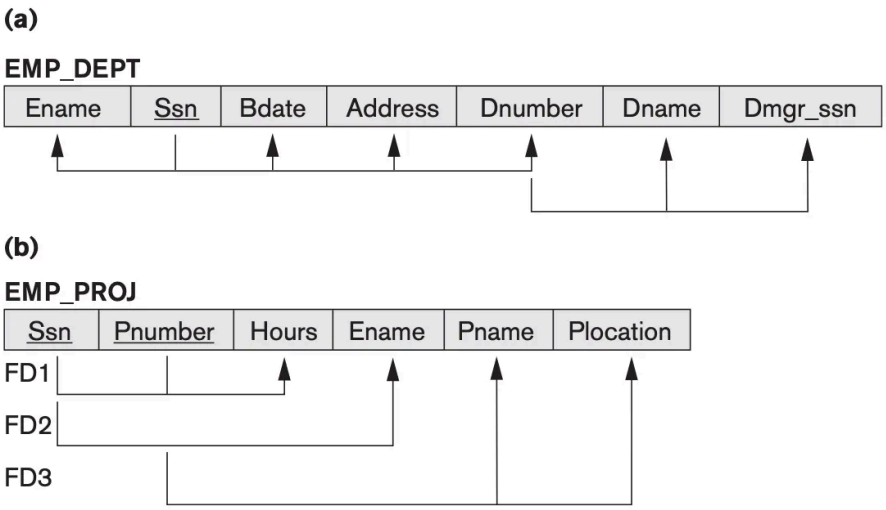
Remember:

- "Might hold" ≠ "does hold."

  Only semantics confirm an FD.
- But "violated" means definitively false.

🎨 Diagram Notation (FD Lines)

FDs are often drawn like:



Vertical lines join the left-hand side attributes; arrows point to dependent attributes.

🧰 The Set F of Functional Dependencies

When designing a schema:

- The designer specifies a set **F** of FDs that are *semantically true*.
- Other FDs may be **derivable** from F.
- These derived FDs matter for *normalization*, discussed in Chapter 16.

FD inference is its own world — Armstrong's axioms, closures, minimal covers — coming soon.

🧼 15.3 Normalization: Cleaning Up Relations (YT Link)

A ritual of removing redundancy, preventing anomalies, and revealing structure.

🌿 15.3.1 What Is Normalization?

Normalization is a **top-down analytical process** that evaluates a relation schema against a hierarchy of **normal forms**—each stricter, cleaner, more anomaly-proof than the last.

🧪 The Core Idea

Take a relation → test it → if it violates a normal form → **decompose** it → continue.

Repeat until the schema passes all required tests.

📜 Historic Roots (short + sweet)

- Introduced by **E. F. Codd (1972)**.
- Started with: **1NF** → **2NF** → **3NF**
- Later improved to **BCNF** (stronger form of 3NF).
- Higher forms came later: **4NF** (multivalued dependencies) and **5NF** (join dependencies).
    - These are rare in real systems.

🎯 Purpose of Normalization

A relational schema should:

1. **Minimize redundancy**
2. **Avoid anomalies** (insert, delete, update)

Normalization acts like a filter — each pass removes more impurities.

🧰 What Normalization Gives You

A formal method to analyze schemas using:

- **Functional dependencies (FDs)**
- **Primary keys**

And apply a structured sequence of normal form tests.

🏆 Definition: Normal Form

> The normal form of a relation = the highest normal form it satisfies.

Example: If a table passes 1NF and 2NF but not 3NF → it's in **2NF**.

🧩 But Normal Forms Alone Aren't Enough

A database isn't just isolated tables — it's a *network*.

When you decompose relations, the resulting schemas **must** satisfy two crucial properties:

1️⃣ Lossless (Nonadditive) Join Property — Mandatory

Decomposition must not produce **spurious tuples**.

When you join the decomposed tables back, you must get **exactly** the original relation.

No lossless join = broken design.

When you split a table into two (or more), you *must* be able to put them back together and recover **exactly** the original rows.

No weird extra rows.

No missing rows.

No "Franken-tuples."

If the join isn't lossless → your schema is broken.

2️⃣ Dependency Preservation — Desirable (but sometimes sacrificed)

All the FDs should remain enforceable **without doing joins.**

If you *lose* dependency preservation but keep lossless join, that's acceptable.

If you keep dependencies but lose lossless join—never acceptable.

Think of dependency preservation as the velvet cushions.

Lossless join is the foundation of the house.

🛠️ 15.3.2 Practical Use

Designers rarely start from scratch — they inherit:

- Legacy DBs
- Old models
- Spreadsheets

Normalization is applied:

- To fix issues
- To clean up relationships
- To remove anomalies

🎯 Most practical schemas normalize up to:

- **3NF**
- or **BCNF**
- sometimes **4NF**

5NF is rarely needed in the real world.

⚠️ Performance Caveat

Sometimes designers **intentionally stop early** for performance:

> Denormalization: storing a table in a lower normal form to avoid expensive joins.

You get speed, but you also get anomalies — your call.

🔐 15.3.3 Keys and Attributes in Normalization

To apply normal forms, we must understand:

- **Keys**
- **Superkeys**
- **Candidate keys**
- **Prime & non-prime attributes**

🔑 Superkey

A set of attributes **S ⊆ R** such that no two tuples in any legal state can have the same values for S.

```
Ssn → uniquely identifies an employee
```

Any supersets of a key are also superkeys.

🗝️ Key (Minimal Superkey)

A superkey **K** is a *key* if it is **minimal**:

> Remove any attribute → it stops being a superkey.

Example:

`{Ssn}` is a key, but `{Ssn, Ename}` is only a superkey.

🎯 Candidate Keys

A relation can have multiple keys:

- All are **candidate keys**
- One is chosen as the **primary key**

Example:

WORKS_ON has `{Ssn, Pnumber}` as a candidate key.

🌟 Prime vs Non-Prime Attributes

- **Prime attribute** → appears in some candidate key
- **Nonprime attribute** → does not appear in *any* candidate key

Example (WORKS_ON):

- Prime: `Ssn`, `Pnumber`
- Nonprime: `Hours`, others

📚 What's Next?

Codd's path:

1NF → 2NF → 3NF → BCNF

Each solves a different type of problem:

- 1NF handles **atomicity**
- 2NF handles **partial dependencies**
- 3NF handles **transitive dependencies**
- BCNF handles **all FD-based anomalies**

And by definition:

> Every 3NF relation is already in 2NF.
>
> Every BCNF relation is in 3NF.

🧱 15.3.4 First Normal Form (1NF)

Where relations stop being wild forests and become neatly pruned gardens.

🌟 What 1NF Really Means

1NF requires that **every attribute of every tuple holds only atomic (indivisible)** values.

No sets.

No lists.

No nested tables.

No tuples inside tuples.

Just *simple, single values*.

💡 Core requirement:

> Attribute domains must contain only atomic values, and each attribute value in each tuple must be a single atomic value.

If your table has “multiple values” in a single cell → it’s not 1NF.

If your attribute holds a set or a structure → still not 1NF.

If a column contains a mini-table → run.

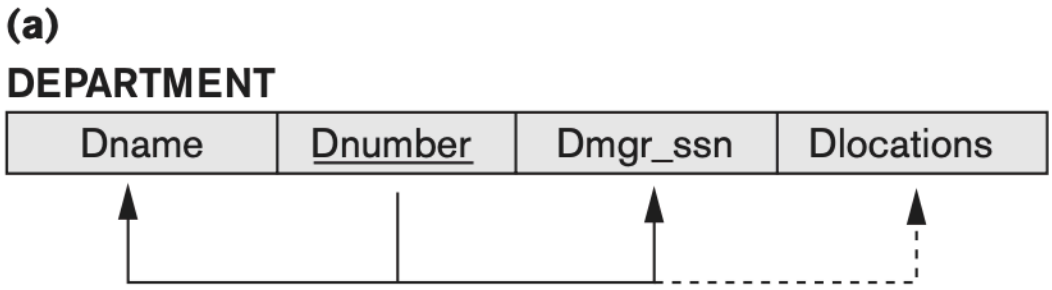
📌 Example: DEPARTMENT (Not in 1NF)

❌ Problem Attribute: Dlocations

Departments can have **multiple locations**, giving rise to representations like:

```
Dlocations = {Bellaire, Sugarland, Houston}
```

Since a single attribute value holds **a set**, the relation is **not in 1NF**.



🧠 Two Semantics for Dlocations

Depending on design intent:

1️⃣ Domain is atomic, but tuples store *sets* → still violates 1NF

- Means: domain allows single locations
- But *tuples* use multiple locations
- FD: `Dnumber` does **not** determine a single Dlocation (violates FD expectations)

2️⃣ Domain itself is a set (non-atomic)

- FD: `Dnumber → Dlocations`
- But attribute values themselves are still **non-atomic** → violates 1NF

Either way, **still not 1NF**.

(b)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

🛠️ How to Fix a Non-1NF Attribute

Three classic strategies:

🥇 Strategy 1 — Decompose into a New Relation (Best Method)

Create a new relation with one location per row:

DEPT_LOCATIONS(Dnumber, Dlocation)

Primary key → `{Dnumber, Dlocation}`

No redundancy, no NULLs, no limit on #locations.

✔️ Most general

✔️ No anomalies

✔️ No arbitrary limits

> This is the preferred and theoretically correct solution.

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

🥈 Strategy 2 — Expand the Key (Duplicate Original Rows)

Repeat the department row for each location:

```
5 | Research | 333445555 | Bellaire
5 | Research | 333445555 | Sugarland
5 | Research | 333445555 | Houston
```

Primary key becomes: **{Dnumber, Dlocation}**

⚠️ Introduces major **redundancy**

⚠️ Repeated department name, manager_ssn, etc.

⚠️ Later normalization will force decomposition anyway

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

🥉 Strategy 3 — Fixed Number of Atomic Attributes

If max #locations = 3, create:

- Dlocation1
- Dlocation2
- Dlocation3

⚠️ Introduces arbitrary limits

⚠️ Causes NULLs

⚠️ Artificial ordering ("Is Bellaire #1 or #2?")

⚠️ Makes queries painful

This is the *hacky spreadsheet solution*, not a database solution.

🧩 Why Strategy 1 Wins

- No redundancy
- No NULL pollution
- No arbitrary constraints
- Clean, intuitive, and scalable

It fits perfectly into the later normal forms (2NF, 3NF, BCNF).

🧱 Nested Relations Also Violate 1NF

Example: EMP_PROJ with embedded PROJS relation:

(a)
EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

EMP_PROJ

Ssn | Ename | PROJS(Pnumber, Hours)

Each tuple contains a *relation* inside it → **nested relation** → violates 1NF.

(b)
EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

To normalize:

Decompose into:

- **EMP_PROJ1(Ssn, Ename)**

- **EMP_PROJ2(Ssn, Pnumber, Hours)**

  (Propagate `Ssn` into the new table; combine with partial key `Pnumber`.)

(c)

EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

This is known as **unnesting**.

🧨 Special Pitfall: Two Multivalued Attributes in One Relation

PERSON (Ss#, {Car_lic#}, {Phone#})

This relation represents the fact that a person has multiple cars and multiple phones

If you normalize by expanding rows (Strategy 2), you get:

PERSON_IN_1NF (Ss#, Car_lic#, Phone#)

This forces *all combinations* of cars × phones → massive redundancy → wrong semantics.

Correct solution:

- **P1(Ss#, Car_lic#)**

- **P2(Ss#, Phone#)**

This anticipates **multivalued dependencies & 4NF**.

🧭 Summary of 1NF Rules

Feature	Allowed in 1NF?	Notes
Atomic attribute values	✅	Required
Multivalued attributes	❌	Must be decomposed
Composite attributes	❌	Must be flattened
Nested relations	❌	Must be unnested
Unlimited multivalued attributes	Only if separated	Use decomposition

---

🔁 15.3.5 Second Normal Form (2NF) — Full vs Partial Dependencies

Fixing the "part of the key" problem.

🧠 Key concept: Full functional dependency

- **X → Y** is **full** if **no proper subset** of **X** determines **Y**.
- If some subset of **X** still determines **Y**, then **X → Y** is a **partial dependency**.

Example:

- `{Ssn, Pnumber} → Hours` is **full** (neither Ssn nor Pnumber alone → Hours).
- `{Ssn, Pnumber} → Ename` is **partial** because `Ssn → Ename`.

✅ Definition: 2NF

A relation **R** is in **Second Normal Form (2NF)** if **every non-prime attribute** (attribute not in any candidate key) is **fully functionally dependent on every key** of R — i.e., **no non-prime attribute** is partially dependent on a candidate key.

- If the primary key is a **single attribute**, 2NF is automatically satisfied.
- 2NF is mainly relevant when the key is **composite** (multiple attributes).

🔧 Remedy when 2NF fails

Decompose R into relations where non-prime attributes attach only to the part of the key they fully depend on.

EMP_PROJ example →

Decompose `EMP_PROJ(Ssn, Pnumber, Hours, Ename, Pname, Plocation)` into:

- `EP1(Ssn, Pnumber, Hours)` — preserves `{Ssn,Pnumber} → Hours`
- `EP2(Ssn, Ename)` — captures `Ssn → Ename`
- `EP3(Pnumber, Pname, Plocation)` — captures `Pnumber → {Pname,Plocation}`

Each of EP1/EP2/EP3 is in **2NF**.

🔁 15.3.6 Third Normal Form (3NF) — Removing transitive ugliness

Fixing the "X → Z → A" chains that hide redundancy.

🧠 Key concept: Transitive dependency

A dependency `X → Y` is **transitive** if `X → Z` and `Z → Y` hold, and `Z` is **not** a key (nor subset of any key). That makes `Y` indirectly depend on `X` through `Z`.

Example:

In `EMP_DEPT`, `Ssn → Dnumber` and `Dnumber → Dmgr_ssn`, so `Ssn → Dmgr_ssn` is **transitive** via `Dnumber`.

✅ Definition: 3NF (original Codd)

A relation is in **3NF** if it is in **2NF** and **no non-prime attribute** is **transitively dependent** on the primary key.

(Equivalent intent: avoid non-prime → non-prime FDs.)

🔧 Remedy when 3NF fails

Decompose to remove the middle attribute(s) of the transitive chain. The goal is that non-prime attrs should depend **directly** on keys only.

EMP_DEPT example →

Decompose `EMP_DEPT` into:

- `ED1(Ename, Ssn, Bdate, Address, Dnumber)` — employee facts
- `ED2(Dnumber, Dname, Dmgr_ssn)` — department facts

A NATURAL JOIN on ED1 ⋈ ED2 recovers the original relation (lossless). Both are in **3NF**.

🧾 15.4 Generalized Definitions (consider all candidate keys)

Don't favor the primary key — treat all candidate keys equally.

🔁 General 2NF

A relation is in **2NF** if **no non-prime attribute** is **partially dependent on any candidate key**.

(Equivalently: fully dependent on **every** key.)

LOTS example (illustrative):

- Candidate keys: `Property_id#` and `{County_name, Lot#}`.
- FDs:
  - `FD1: Property_id# → (other attrs)`
  - `FD3: County_name → Tax_rate` ← **partial** relative to `{County_name, Lot#}`
  - `FD4: Area → Price` ← independent FD

Decompose LOTS into:

- `LOTS1(Property_id#, County_name, Lot#, Area, Price)`
- `LOTS2(County_name, Tax_rate)`

  Now LOTS1 and LOTS2 are in **2NF**.

🔁 General 3NF (clean formal test)

A relation **R** is in **3NF** iff for every **nontrivial** FD `X → A` that holds in R, **either**:

- (a) **X is a superkey** of R, **or**
- (b) **A is a prime attribute** (i.e., part of *some* candidate key).

This formulation avoids needing to check 2NF first — apply directly to all FDs and candidate keys.

🔍 Interpreting the 3NF rule

Violations of 3NF typically fall into two buckets:

1. A **non-prime attribute** determines another **non-prime attribute** (transitive problem).
2. A **proper subset** of a key determines a non-prime attribute (partial dependency).

Alternate phrasing: every non-prime attribute must be:

- fully dependent on every key, and
- non-transitively dependent on every key.

📌 Quick practical checklist — Normalization workflow

1. Ensure **1NF**: atomic attributes only.
2. Identify **all candidate keys** and **prime/non-prime attributes**.

3. Test for **2NF**: remove partial dependencies (decompose).
4. Test for **3NF**: remove transitive dependencies (decompose).
5. Verify **lossless join** (must) and **dependency preservation** (desirable).
6. If needed and meaningful, consider **BCNF**, **4NF**, **5NF** (advanced & rarer).

⚠️ Practical notes & trade-offs

- **Normalization reduces redundancy and anomalies**, but can increase the number of joins (performance trade-off).
- **Denormalization** (intentional lower-normal-form storage) is sometimes used for performance — but accept anomalies risk.
- Real-world designs often target **3NF/BCNF**; 4NF/5NF are niche.

---

rules of inference:

reflexive: if Y is a subset of X, then $X \rightarrow Y$

augmentation: if $X \rightarrow Y$, then $XZ \rightarrow YZ$

transitive: if $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$

decomposition: $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$

union: $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$

psuedo-transitivity: $X \rightarrow Y$ and $WY \rightarrow Z$ then $WX \rightarrow Z$